

Unit 1

Principles of object Oriented Programming

C++ was developed by Bjarne Stroustrup at Bell Labs in New Jersey in 1980.

- C++ is an object oriented programming language.
- C++ is a superset of C
- Stroustrup, an admirer of Simula 67 & a strong supporter of C, wanted to combine the best of both the lang. & create a more powerful lang. that support object oriented prog.
- C++ is an extension of C with a major addition of the class construct feature of Simula 67.

Class was a major addition in C. Stroustrup initially called the new lang. 'C with classes'. Later in 1983, the name was changed to C++. The idea of C++ comes from the C increment operator ++, thereby suggesting that C++.

* C++ is a lot of well structured & safer lang. than C as it OOP's based.

C++ is a general purpose, object oriented prog. lang. most widely used in famous S/w. & appliⁿ.

eg. Airline ticketing, Amazon (web commerce), Google (web search), Facebook (social media), database access.

Diff. betⁿ procedure oriented programming & object oriented programming.

POP

OOP

- | | |
|---|--|
| 1) In POP, prog. is divided into small parts called functions. | 1) In OOP, prog. is divided into parts called objects. |
| 2) Importance is not given to data but only sequence of actions to be done. | 2) Imp. is given to the data rather than procedure or bcoz it works on real world. |
| 3) POP follows Top-down approach. | 3) OOP follows bottom up approach. |
| 4) POP does not have any access specifiers. | 4) OOP has access specifiers named public, Private, Protected. |
| 5) POP does not have any proper way for hiding data so it is less secure. | 5) OOP provides data hiding so it is more secure. |
| 6) In POP, overloading is not possible. | 6) In OOP, overloading is possible in the form of fun ⁿ & operator. |
| eg. C, VB, FORTRAN, PASCAL. | eg. C++, Java, C#, VB.NET, python & JavaScript. |

Basic concepts of OOP :-

- ① C++ variables
- variables are the backbone of any prog. lang.
 - A variable is a way to store some info. for later use. We can retrieve this value or data by referring to a "word" that will describe this info.

- Once declared & defined they may be used many times within the scope in which they were declared.

② C++ control structures +

- when a prg. runs, the code is read by the compiler line by line. This is known as 'code flow'.
- when the code is being read from top to bottom it may encounter a point where it needs to make a decision. Based on the decision, the prg. may jump to diff. part of the code.

③ C++ data structures +

lets suppose in college list of courses are more. but how will we store them as diff. users may register for diff. courses. we will not generate diff variable for each user.

So best way for storing diff. variables in data structure.

D.S. is a great way to get around having to create thousands of variables. C++ contains many types of inbuilt d.s.

④ C++ Syntax +

The syntax is a layout of words,

⑤ C++ expressions & symbols.

C++ tools +

IDE tool is s/w. makes coding easier.

Characteristics of object oriented prog. + 2)

Basic concepts of OOP +

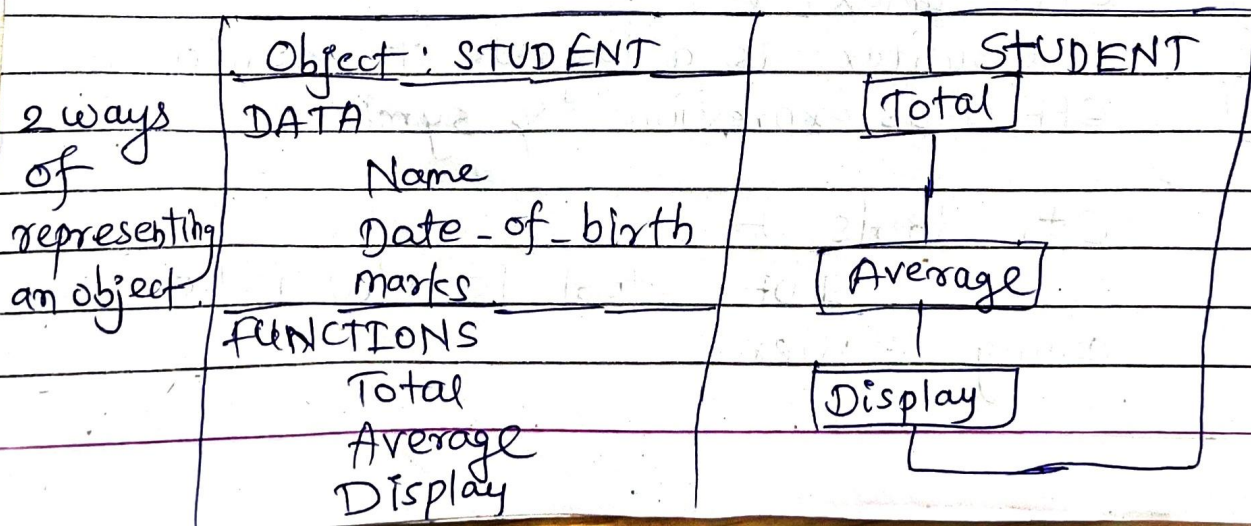
- 1) objects
- 2) classes
- 3) Data abstraction &
- 4) Encapsulation
- 5) Inheritance
- 6) polymorphism
- 7) Dynamic binding
- 8) message passing

1) object +

Objects are the run time entities in an object oriented system. They may represent a person, a place, a bank account, a table of data or any item that the prog. has to handle.

- They may also represent user-defined data such as vectors, time & list.
- When a prog. is executed, the objects interact by sending messages to one another.

eg. If customer & account are two objects in a prog. then the customer object may send msg. to account object for bank balance.



2) Classes +

A class in C++ is a user defined type or data structure declared with keyword class that has data & functions (methods) as its members whose access is governed by the 3 access specifier private, public, protected.

* Private members are not accessible outside the class, they can be accessed only through methods of the class.

* Public members form an interface to the class & are accessible outside the class.

3) Data abstraction & Encapsulation +

The wrapping up of data & functions into a single unit is known as encapsulation. also known as "data-hiding".

- Data is not accessible to the outside world & only those functions which are wrapped in the class can access it.
- These function provide the interface betⁿ the objects data & the program.

* Abstraction refers to the art of representing essential features without including the background details or explanations.

4) Inheritance +

Inheritance is the process by which objects of one class acquire the properties of objects of another class.

- It supports the concept of hierarchical classification.

eg. The bird 'robin' is a part of the class 'flying bird' which is again part of the class bird.

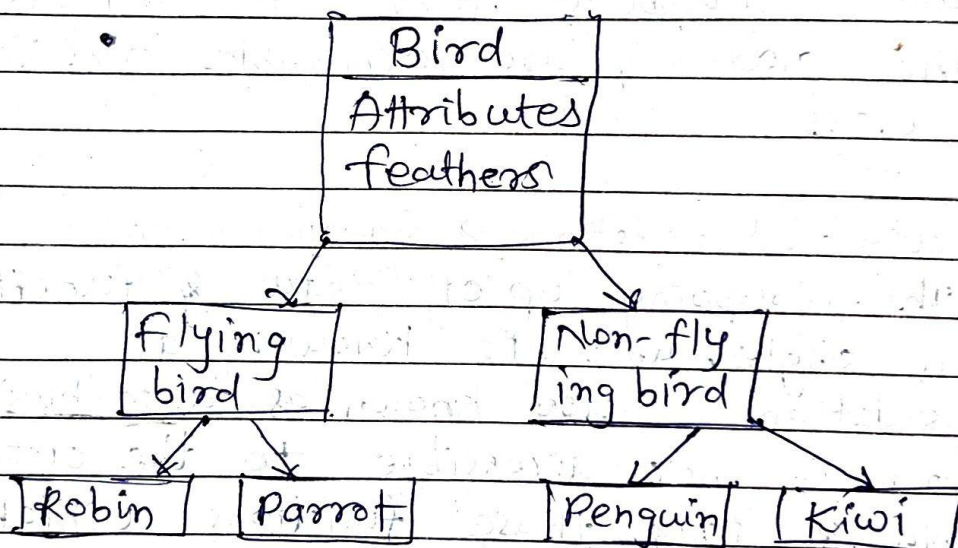
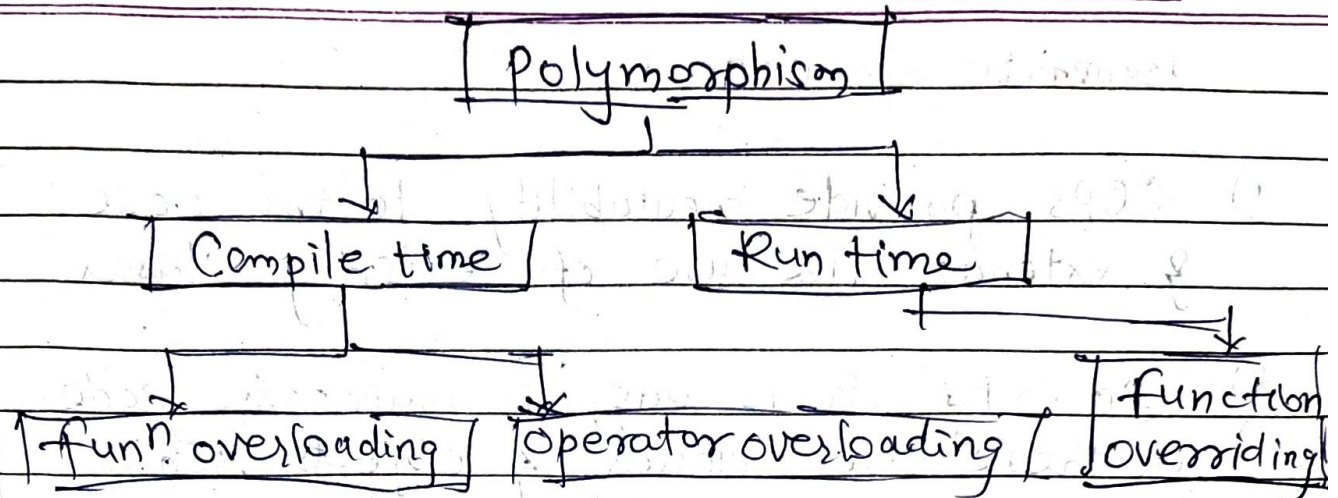


fig. Property Inheritance

5) Polymorphism +

polymorphism means many forms. It is the ability to take more than one form.

Polymorphism real-life eg. a person at the same time can have diff. roles to play in life. Like a woman, at the same time is a mother, a wife, an employee & a daughter.



6) Dynamic binding +

The implementation of the method that is used on a particular call is determined dynamically at runtime. This capability is known as dynamic binding in OOP.

* In simple terms, Dynamic binding is the connection betⁿ the funⁿ declaration & the function call.

* It takes place using virtual function.

* It takes place at runtime so it is referred to as Late binding.

real objects use dynamic binding.

virtual functions +

A virtual function is a member funⁿ declared in a base class & re-declared in a derived class (overridden).

You can execute the virtual funⁿ of the derived class when you refer to its object using a pointer or reference to the base class. The concept of dynamic binding is implemented with the help of virtual functions.

Benefits of oops ↴

- 1) OOPS provide reusability to the code & extends the use of existing classes.
- 2) In OOPS it is easy to maintain code as there are classes & objects, which helps in making it easy to maintain rather than restructuring.
- 3) It also helps in data hiding, keeping the data & information safe from leaking or getting exposed.
- 4) OOP is easy to implement & maintain.
- 5) Programing is more readable, having better performance.
- 6) OOP is an open-source lang. prog.
- 7) Encapsulation, Inheritance, Polymorphism, abstraction, modularity, code reusability, extendibility, easy debugging etc.

7) message passing ↴

It is a form of commuⁿ. used in oop. objects communicate with one another by sending or receiving info. to each other.

A msg. for an object is a request for execution of a procedure & therefore will invoke a funⁿ in the receiving object that generates the desired result.

* The most important facilities that C++ adds on to C are classes, inheritance, funⁿ, overloading & operator overloading.

* Those features enable creating of abstract data types, inherit properties from existing data types & support polymorphism thereby making C++ a truly object-oriented lang.

GNU compiler collection.

S/w for C++ → Turbo C, GCC, Eclipse, CodeBlock, NetBeans etc.

Structure of C++ →

* C++ program that prints a string on the screen.

* Save the file with extension cpp
(file_name.cpp) : program 1.cpp

```
#include <iostream.h>
```

```
using namespace std;
```

```
int main () {
```

```
    cout << "Welcome to SY IT"; //output
```

```
    return 0;
```

```
}
```

Comments →

* C++ introduces a new comment symbol // (double slash). Comments start with a double slash symbol & terminate at the end of the line.

* A comment may start anywhere in the line, & whatever follows till the end of the line is ignored.

```
// SY
// IT
// GHRCEM
```

C++ program for addition of 2 nos.

program 2.cpp

```
#include <iostream.h>
```

```
using namespace std;
```

```
int main ()
```

```
{
```

```
    int a, b, c;
```

```
    cout << "Enter the value of a & b";
```

```
    cin >> a;
```

```
    cin >> b;
```

```
    c = a + b;
```

```
    cout << "Addition=" << c << "\n";
```

```
    return 0;
```

```
}
```

Output Operator +

```
cout << "Enter the value of a and b";
```

==== Above statement causes the string in quotation marks to be displayed on the screen.

==== This statement introduces two new C++ features, cout and <<.

- * The identifier `cout` is a predefined object that represents the standard output stream in C++.
- * The operator `<<` is called the insertion or put to operator. It inserts (or sends) the contents of the variable on its right to the object on its left.

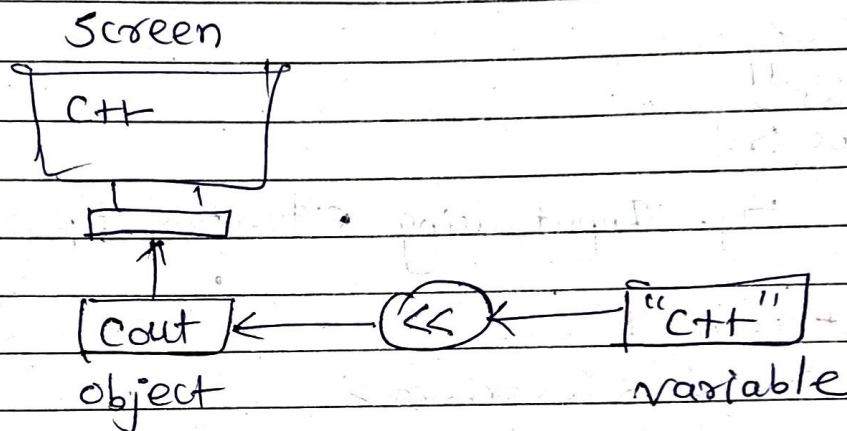


fig. Output using Insertion operator.

Input Operator $\rightarrow$

- * `Cin, >>` these are new features introduced in C++ for input purpose.
- * The identifier `cin` is a predefined object in C++ that corresponds to the standard input stream.

`Cin >> a ;`

- Is an input statement & causes the prog. to wait for the user to type in a number.
- The number keyed in is placed in the variable number `a`.

- * The operator `>>` is known as extraction or get from operator. It extracts (or takes) the value from the keyboard and assigns it to the variable on its right.

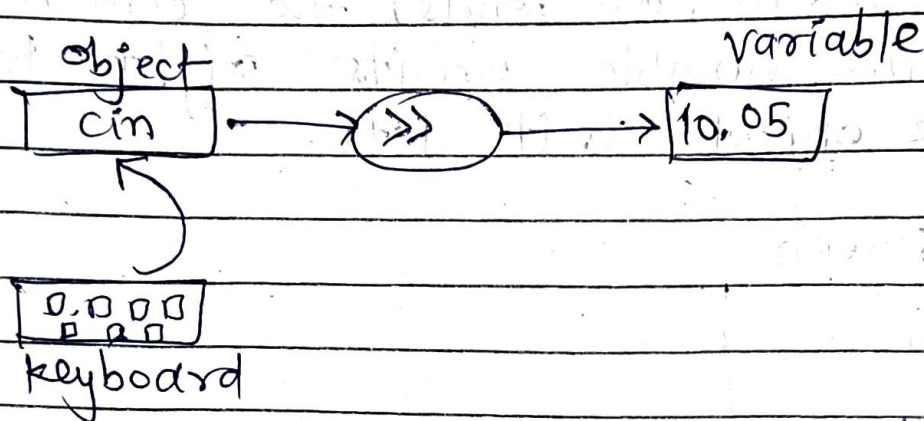


fig. Input using extraction operator.

iostream +

```
#include <iostream>
```

- * This directory causes the preprocessor to add the contents of the `iostream` file to the program.
- * It contains declarations for the identifier `cout` & the operator `<<`.
- * The header file `iostream` should be included at the beginning of all programs that use input/output stmt.

Keywords +

- * They are explicitly reserved identifiers.
- * and cannot be used as names for the prog. variables or other user-defined prog. elements.
- * The keywords implement specific C++ lang. features.

Cascading of I/O operators

```
cout << "Addition =" << c << "\n";
```

- * first sends the string "Addition =" to cout & then sends the value of c.
- * finally ,
it sends the newline character so that the next op will be in the new line.
- * The multiple use of << in one statement is called cascading.
- * When cascading an output operator, we should ensure necessary blank spaces betⁿ diff. items.

Namespace

```
#include <iostream>
```

- * Namespace is a new concept introduced by the ANSI C++ standard committee.
- * This defines a scope for the identifiers that are used in a program.
- * for using the identifiers defined in the namespace scope we must include the using directive, like
using namespace std;
- * Here, std is the namespace where ANSI C++ standard class libraries are defined.
- * All ANSIC++ prog. must include this directive.

* This will bring all the identifiers defined in std to the current global scope.

using & namespace are the new keywords of C++.

Return type of main()

* In C++, main() returns an integer type value to the operating system.

* Therefore, every main() in C++ should end with a return(0) statement; otherwise a warning or error might occur.

* Since main() returns an integer type value, return type for main() is explicitly specified as int.

* It indicates that program has been executed successfully & terminate the function. (exit status)

* Note that the default return type for all functions in C++ is int.

Token

* The smallest individual units in a prg. are known as tokens.

* A token is the smallest element of a prg. that is meaningful to the compiler.

* C++ has the following tokens :

keywords

Identifiers

constants

Strings

Operators

special symbol.

① Keywords +

* The keywords implement specific C++ language features.

* They are explicitly reserved identifiers.

* and cannot be used as names for the prg. variables or other user-defined program elements.

② Identifiers +

* Identifiers refer to the names of variables, functions, arrays, classes etc. created by the programmer.

* They are the fundamental requirement of any lang.

* Each lang. has its own rules for naming these identifiers.

④ String +

- * Strings are nothing but an array of characters ended with a null character ('\0').
- * This null character indicates the end of the string.
- * Strings are always enclosed in double quotes.
- * Whereas, a character is enclosed in single quotes in C and C++.

Declarations for string:

```
char string1[20] = {'s', 'y', 'i', 't', 'g', 'h', 'r', 'c',  
                  'e', 'm', '\0'};
```

```
char string2[20] = "syitghrcem";
```

```
char string3[] = "syitghrcem";
```

⑤ Operators +

- * C++ has a rich set of operators.
- * All C operators are valid in C++.
- * Operators are symbols that trigger an action when applied to C++ variables & other objects.
- * The data items on which operators act upon are called operands.

* Depending on the number of operands that an operator can act upon, operators can be classified as follows:

Unary Those operators that require only single operand to act upon are known as unary operators.

eg. -1 , $-b$, $+z$.

- Uppercase & lowercase letters
- A declared keyword cannot be used as a variable name.

* A major difference betⁿ C & C++ is the limit on the length of a name.

* While ANSI C recognizes only the first 32 characters in a name.

* ANSI C++ places no limit on its length & therefore, all the characters in a name are significant.

eg.

```
void main()
{
    int a = 10;
}
```

In the above prg. there are 2 identifiers:

1. main: method name.
2. a: variable name.

③ Constants +

- * constants refer to fixed values that do not change during the execution of a prg.
- * C++ supports several kinds of literal constants.
- * They include integers, characters, floating point no. & strings.
- * Literal constant do not have memory locations.

Binary Those operators that require two operands to act upon are called binary operators.

Binary operators are classified into +

- ① Arithmetic operators $\rightarrow + - * /$
- ② Relational operators $\rightarrow ==, >=, <=$
- ③ Logical operators $\rightarrow \&\&, ||$
- ④ Assignment operators $\rightarrow = += -=$
- ⑤ Conditional operators $\rightarrow ? : (Ex: (n_1 > n_2) ? n_1 : n_2)$
(ternary opr.)
- ⑥ Bitwise operator $\rightarrow \&, ^, \ll, \gg$
- ⑦ Insertion operator $\rightarrow \ll$
- ⑧ Extraction operator $\rightarrow \gg$
- ⑨ the other new operators are :

:: $\rightarrow$ Scope resolution operator.

::* $\rightarrow$ pointer-to-member declarator.

$\rightarrow^*$ $\rightarrow$ pointer-to-member operator.

.* $\rightarrow$ pointer-to-member operator.

delete $\rightarrow$ memory release operator.

endl $\rightarrow$ line feed operator.

new $\rightarrow$ memory allocation operator.

setw $\rightarrow$ field width operator.

* like C, C++ is also a block-structured language.

* Blocks & scope can be used in constructing programs.

* The scope of the variable extends from the point of its declaration till the end of the block containing the declaration.

* A variable declared inside a block is said to be local to that block. consider the following segment of program

* The two declarations of x refer to two diff. memory locations containing diff. values.

* Statements in the second block cannot refer to the variables declared in the first block & vice versa.

* Blocks in C++ are often nested.

* Block 2 is contained in block 1.

* Within the inner block, the variable x will refer to the data object declared there in.

* A data object created inside a block with `new`, will remain in existence until it is explicitly destroyed by using `delete`.

* The `new` operator can be used to create objects of any type.

eg. `pointer_variable = new data_type;`
`p = new int;`
`q = new float;`

* When a data object is no longer needed, then it is destroyed to release the memory space for reuse.

eg. `delete pointer_variable`
`delete p;`
`delete q;`

endl manipulator

* When used in an output statement causes a Line feed to be inserted.

* It has the same effect as using the newline character "\n".

```
cout << "m = " << m << endl;
```

setw

```
cout << setw(5) << sum << endl;
```

* The manipulator `setw(5)` specifies a field width 5 for printing the value of the variable `sum`.

* This value is right-justified within the field as shown below.

If `sum = 235`.

5 4 3 2 1
2 3 5
1 2 3 4 5

Function Overloading

It is a feature of OOP where two or more functions can have the same name but diff. parameters.

When a function name is overloaded with diff. jobs it is called funⁿ overl.

F.O. can be eg. of polymorphism in C++.

(a) Parameters should have a diff. ~~type~~ number.

```
add(int a, int b)
```

```
add(int a, int b, int c)
```

eg.

```
#include <iostream>
```

```
using namespace std;
```

```
void add(int a, int b)
```

```
{
```

```
    cout << "sum = " << (a+b);
```

```
}
```

```
void add(int a, int b, int c)
```

```
{
```

```
    cout << endl << "sum = " << (a+b+c);
```

```
}
```

```
int main()
```

```
{
```

```
    int add(10, 2);
```

```
    add(5, 6, 4);
```

```
    return 0;
```

```
}
```

O/p

```
sum = 12
```

```
sum = 15
```

(b) Parameters should have diff. sequence of para.

(c) Parameters should have diff type.

Operator overloading +

* operator overloading is a compile-time polymorphism.

* In C++, we can make operators work for user-defined classes.

* C++ has the ability to provide the operators with a special meaning without changing its original meaning. It is called operator overloading.

eg. we can overload an operator '+' in a class like String so that we can concatenate two strings by just using +.

```
int a;
```

```
float b, sum;
```

```
sum = a + b;
```

* Here variables 'a' and 'b' are of types "int" and "float", which are built-in data types.

* Hence the addition operator '+' can easily add the contents of 'a' & 'b'. This is because the addition operator "+" is predefined to add variables of built-in-data type only.

* Function overloading means using a single name & giving more functionality to it. Operator overloading means adding extra functionality for a certain operator. When an operator is overloaded, the operator has different meanings, which depend on the type of its operands.

Constructor ↴

A constructor is a member function of a class that has the same name as the class name.

- * It helps to initialize the object of a class.
- * It can either accept the arguments or not. It is used to allocate the mem. to an object of the class.
- * It is called whenever an instance of the class is created.
- * It can be defined manually with arguments or without arguments.
- * There can be many constructors in a class.
- * There is a concept of copy constructor which is used to initialize an object from another object.

Syntax ↴

```
class name ()
```

```
{
```

```
// constructor body
```

```
}
```

Destructor ↴

- * A destructor is also a member fun. of a class that has the same name as the class name preceded by tilde (~) operator.
- * It also helps to deallocate the mem. of an object.
- * In a class there is always a single destructor without any parameters so it can't be overloaded.

* If a class is inherited by another class and both the classes have a destructor then the destructor of the child class is called first, followed by the destructor of the parent or base class.

Syntax

```
~ class name ()
```

```
{
```

```
    // destructors body
```

```
}
```

Difference constructor and Destructor characteristics

- | | |
|---|--|
| <p>1) Constructor helps to initialize the object of a class.</p> <p>2) It is declared as <code>className()</code> (arguments if any) { constructors body }</p> <p>3) constructor can either accept arguments or not.</p> <p>4) A constructor is called when an instance or object of a class is created.</p> <p>5) constructor is used to allocate the memory to an instance or object.</p> <p>6) constructor can be overloaded.</p> <p>7) The constructor's name is same as the class name.</p> <p>8) In a class, there can be multiple constructors.</p> <p>9) There is a concept of copy constructor which is used to initialize an object from another obj.</p> <p>10) They are often called in successive order.</p> | <p>1) Destructor is used to destroy the instances.</p> <p>2) It is declared as <code>~className()</code> (no arguments) { }</p> <p>3) while it can't have any arguments.</p> <p>4) It is called while object of the class is freed or deleted.</p> <p>5) it is used to deallocate the mem of an object or instance.</p> <p>6) Destructor cannot be overloaded.</p> <p>7) Destructors name is also same as the class name preceded by tilde (~) operator.</p> <p>8) while in a class, there is always a single destructor.</p> <p>9) there is no copy destructor concept.</p> <p>10) They are called in reverse order of constructor.</p> |
|---|--|